TEC-UCT
INSTITUTO TECNOLÓGICO
UNIVERSIDAD CATÓLICA DE TEMUCO

EVALUACIÓN INTEGRADA N°2

Trabajo de investigación y benchmarking

Sistema de gestión de reservas — ReservasUCT

INTEGRANTES

Christopher Córdova
Anthony García
Felipe Velásquez
Franko Zambrano

DOCENTE	ASIGNATURA	FECHA
Giovanni Ganiffo	Desarrollo Móvil	08/11/2026

1. Introducción y benchmarking

Durante el semestre construimos ReservasUCT, una app en KivyMD pensada para la Academia de Artes Musicales de la UCT. El problema de fondo es simple: hoy la secretaria anota a mano quién usa cada sala y a qué hora, y eso le quita mucho tiempo y se presta para errores. Con la app, un estudiante elige el día en un calendario, completa un formulario con la sala, la hora, el motivo y cuántos acompañantes lleva, y después puede ver o anular lo que reservó.

Para este trabajo quisimos mirar cómo resuelven este mismo problema aplicaciones que ya están en el mercado. Elegimos dos: **Skedda**, que se dedica a la reserva de espacios físicos, y **Calendly**, que se dedica a agendar horarios. Nos pareció una buena combinación porque nuestra app tiene un poco de cada una: se reserva una sala (como en Skedda) en un bloque de hora específico (como en Calendly).

1.1 Skedda

Skedda es una plataforma para que una organización publique sus espacios y la gente los reserve sola, sin pasar por un encargado. Funciona en el navegador y también tiene app para Android e iOS. Lo que la hace atractiva es su calendario en forma de grilla, con una columna por sala y las horas hacia abajo, que permite ver de un vistazo qué está libre. El administrador además puede definir reglas, como la duración máxima de una reserva o con cuántos días de anticipación se puede reservar, y el sistema se encarga de que nadie tome una sala que ya está ocupada.

Apunta a lugares con espacios compartidos: colegios, universidades, coworkings, recintos deportivos y salas de ensayo, entre otros. Tiene dos tipos de usuario bien marcados: quien administra los espacios y quien reserva.

1.2 Calendly

Calendly sirve para coordinar reuniones sin el típico ida y vuelta de correos. Cada persona comparte un enlace con su disponibilidad y el otro escoge el horario que le acomode. Se conecta con Google Calendar u Outlook, así que solo ofrece las horas que de verdad están libres. Un detalle que nos llamó la atención es que quien agenda no necesita crear una cuenta: basta con poner su nombre y correo, y le llega una confirmación con la opción de reprogramar o cancelar.

Está pensada para profesionales que agendan muchas reuniones: equipos de ventas, reclutadores, consultores y también profesores que ofrecen horarios de atención.

1.3 Comparación desde la experiencia de usuario

Aspecto	Skedda	Calendly
Qué se reserva	Un espacio físico: sala, cancha, escritorio	Un horario con una persona
Pantalla principal	Grilla con todas las salas y horas	Calendario del mes y lista de horas libres
¿Hay que tener cuenta?	Normalmente sí, depende de cómo lo configure el administrador	No, basta con nombre y correo
Lo mejor	Muestra la ocupación completa; reglas automáticas que evitan choques	Muy pocos pasos; solo ofrece horas libres; se sincroniza con el calendario
Lo que cuesta	La grilla se ve apretada en pantallas chicas; configurarla toma tiempo; varias funciones son de pago	No maneja espacios ni capacidad de personas; el plan gratis es limitado; quien agenda no ve el panorama general

Dicho en simple: Skedda es más completa para administrar salas, pero tiene una curva de aprendizaje más larga. Calendly es facilísima para el usuario final porque lo lleva de la mano, aunque no está pensada para controlar espacios físicos, que es justo lo que necesita la academia.

2. Desglose arquitectónico (MVC y separación de responsabilidades)

En la Clase 23 vimos que el patrón MVC divide una aplicación en tres partes: el **Modelo**, que guarda los datos y sus reglas; la **Vista**, que es lo que ve el usuario; y el **Controlador** o los servicios, que reciben las acciones y conectan lo uno con lo otro. La separación de responsabilidades apunta a que cada parte se preocupe solo de lo suyo. Como no tenemos acceso al código de Skedda ni de Calendly, lo que sigue mezcla lo que encontramos investigando con lo que se puede deducir usándolas.

2.1 La capa de presentación (Vista)

Las dos apps arman sus pantallas con piezas que se repiten: un selector de fecha, bloques de horario, formularios y botones de confirmar o cancelar. En Skedda todo gira en torno a la grilla; al tocar un bloque libre se abre el formulario. En Calendly el recorrido es en línea recta: día, hora, datos y confirmación.

Lo interesante es que la vista no decide qué horario está libre. Solo dibuja lo que le contesta el servidor y registra lo que toca el usuario. Por eso, si otra persona reserva un bloque, la pantalla se actualiza y ese bloque aparece ocupado sin que la app tenga que "pensar" nada.

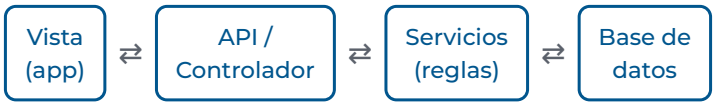
2.2 La lógica de negocio (Controlador y servicios)

Las reglas importantes viven en el servidor y no en el teléfono. En Skedda, por ejemplo, el servidor revisa que la sala no esté tomada, que la reserva no pase de la duración permitida, que se haga con la anticipación mínima y, si el espacio es pagado, calcula el cobro. En Calendly el cálculo fuerte es la disponibilidad: cruza el calendario de la persona, la zona horaria de cada uno, el descanso entre reuniones y el máximo de reuniones por día.

Algunas validaciones sencillas sí se hacen en la app, como revisar que el correo tenga buen formato o que no queden campos vacíos, para avisarle al usuario al tiro. Pero la palabra final la tiene siempre el servidor, porque dos personas pueden intentar tomar el mismo bloque al mismo tiempo desde celulares distintos, y solo el servidor sabe quién llegó primero.

2.3 Cómo se conecta la interfaz con los datos

El recorrido que deducimos es este:



La app nunca toca la base de datos directamente. Le hace una petición a la API del servidor por internet (normalmente en formato JSON), el servidor aplica las reglas, consulta o guarda en la base de datos y devuelve una respuesta que la vista muestra. Es el mismo MVC que vimos en clases, solo que repartido entre el teléfono y la nube.

2.4 ¿Y en nuestra app?

En ReservasUCT seguimos la misma idea, aunque todo corre dentro del mismo programa:

Capa	Archivo	Qué hace
Vista	`views/*.py`	Calendario, formulario, mis reservas y perfil
Modelo	`models/reserva_model.py`	Clase `Reserva`: `validar()` y `choca_con()`
Servicios	`services/gestor.py`	Clase `GestorReservas`: agregar y anular
Datos	`services/persistencia.py`	Guardar y cargar el archivo JSON

Esto nos ha servido harto en la práctica. La regla de que dos reservas no pueden chocar está escrita en un solo lugar (`choca_con`), y las pantallas no necesitan saber cómo se guarda el archivo. Cuando cambiamos la persistencia a JSON, casi no tuvimos que tocar las vistas.

3. Autenticación, seguridad y persistencia

3.1 Cómo gestionan el acceso

Skedda tiene una pantalla de inicio de sesión con correo y contraseña, y recuperación de contraseña por correo. En sus planes más altos ofrece inicio de sesión único (SSO) con Google, Microsoft u otros proveedores que use la organización. También maneja roles: el administrador configura salas y reglas, y el resto solo puede reservar según los permisos que se le den.

Calendly deja entrar al dueño de la cuenta con Google, Microsoft o correo y contraseña. Lo curioso es que quien agenda no necesita cuenta: al confirmar le llega un correo con un enlace único para reprogramar o cancelar, y ese enlace funciona como su "llave" para tocar solo su reserva y no la de otros.

En ambos casos la comunicación viaja cifrada por HTTPS. Una vez que el usuario inicia sesión, la app guarda un token para no pedirle la contraseña cada vez que la abre.

3.2 Cómo manejan la persistencia

Las dos guardan la información en la nube, en una base de datos central. En una app de reservas esto es casi obligatorio: todos tienen que ver lo mismo y al mismo tiempo. Si cada celular tuviera su propia copia, dos personas podrían reservar la misma sala sin enterarse.

Por esa misma razón no son del todo *Offline First*. Es probable que las apps móviles guarden en caché los últimos datos para mostrarlos rápido o cuando no hay señal, pero para crear o cancelar una reserva sí piden conexión. Reservar con información desactualizada terminaría en choques.

3.3 Dónde está parada nuestra app

Hay que reconocer que ReservasUCT todavía no tiene inicio de sesión. La vista de Perfil sigue en construcción, y el nombre y el correo se piden en cada reserva con una validación básica (que el correo tenga "@" y un punto). Lo que sí logramos fue lo siguiente:

- Las reservas se guardan en `reserva/reservas.json` usando la librería `json`. Se cargan al abrir la app y se vuelven a escribir cada vez que alguien agrega o anula una reserva.

- Cada reserva queda con su fecha y hora de registro, gracias a `datetime`.
- Al confirmar, la app genera un código de seis dígitos con `random` y revisa que no se repita. Es una idea bastante parecida al enlace único de Calendly.

La limitación más grande es que ese JSON queda guardado en el dispositivo. Si cada estudiante instala la app en su celular, nadie vería las reservas de los demás. Para que funcione de verdad en la academia habría que pasar a una base de datos compartida en un servidor, tal como hacen Skedda y Calendly.

4. Seguimiento, analítica y métricas

4.1 Lo que hacen las apps analizadas

Por las funciones que ofrecen y por lo que dicen sus políticas de privacidad, que mencionan la recolección de datos de uso y del dispositivo, se nota que hacen seguimiento en dos frentes.

El primero es el que ve el cliente. Skedda le entrega al administrador reportes de uso: qué salas se ocupan más, en qué horarios y quién reserva. Calendly, en sus planes pagados, muestra cuántas reuniones se agendaron, cuántas se cancelaron y cuáles son los horarios más pedidos.

El segundo es interno, y aquí ya hablamos de deducción. Lo normal en apps de este tamaño es registrar la última conexión de cada usuario, los eventos de uso (abrir la app, crear una reserva, cancelarla), el modelo del dispositivo y la versión instalada, además de los errores y cierres inesperados. Para esto se suelen usar herramientas como Firebase Analytics, Firebase Crashlytics o Sentry. A eso se suman los correos de confirmación, los recordatorios y las notificaciones push, que ayudan al usuario y de paso le muestran a la empresa si esa persona sigue activa.

4.2 Cómo lo llevaríamos a ReservasUCT

Como ya trabajamos con JSON y `datetime`, podríamos armar un seguimiento básico sin instalar nada extra. Al abrir la app guardaríamos la fecha y hora de la última conexión. Crearíamos un servicio nuevo, por ejemplo `services/registro.py`, que anote cada evento importante, y lo llamaríamos desde `GestorReservas` al agregar o anular. Los errores al leer el archivo los atraparíamos con `try/except` y los dejaríamos en un log con la librería `logging` (Kivy ya genera sus propios logs, y de hecho nos sirvieron para depurar). Con todo eso, más lo que ya está en `reservas.json`, la secretaria podría saber qué sala se pide más, en qué horarios hay más demanda y cuántas reservas se anulan.

Así quedaría el registro de eventos, siguiendo el mismo estilo de `persistencia.py`:

```
import json, os
from datetime import datetime

ARCHIVO_EVENTOS = "reserva/eventos.json"

def registrar_evento(tipo, detalle=""):
    eventos = []
    if os.path.exists(ARCHIVO_EVENTOS):
        with open(ARCHIVO_EVENTOS, "r", encoding="utf-8") as archivo:
            eventos = json.load(archivo)
    eventos.append({
        "evento": tipo,      # ej: "reserva_creada", "reserva_anulada"
        "detalle": detalle,
        "fecha": datetime.now().strftime("%d/%m/%Y %H:%M:%S")
    })
    with open(ARCHIVO_EVENTOS, "w", encoding="utf-8") as archivo:
        json.dump(eventos, archivo, ensure_ascii=False, indent=4)
```

Si algún día la app llega a Android, lo lógico sería cambiar esto por Firebase Analytics y Crashlytics, que hacen lo mismo pero mandan los datos a la nube.

5. Conclusiones y lecciones para nuestro proyecto

5.1 Lo que copiaríamos o adaptaríamos

- **Mostrar solo las horas libres.** Hoy el formulario deja elegir cualquier hora y recién al confirmar avisa que hay choque. Sería mucho más cómodo que el menú ofrezca solo las horas disponibles para esa sala y ese día, como hace Calendly.
- **Una grilla por sala para la secretaria.** Para el panel administrativo que tenemos pendiente, una vista con las salas en columnas y las horas en filas, al estilo de Skedda, sería la forma más clara de ver cómo viene la semana.
- **Usar el código de acceso para anular.** Ya lo generamos, pero todavía no cumple ninguna función. Si lo pidiéramos al anular, nadie podría cancelar la reserva de otra persona.
- **Reglas fáciles de cambiar.** En parte ya lo hacemos con `SALAS`, `HORAS` y `MAX_ACOMPANANTES` en el modelo, y se podría extender a la duración máxima o a la anticipación mínima.
- **Recordatorios.** Un aviso antes de la hora reservada para que nadie se olvide de su sala.

5.2 Errores o limitaciones que evitamos

Skedda tiene tantas opciones que configurarla toma su tiempo; nuestra app, en cambio, se recorre en tres pasos: calendario, formulario y mis reservas. Calendly no sabe nada de salas ni de capacidad, mientras que nosotros validamos la sala y el máximo de acompañantes. Nuestro calendario además bloquea las fechas pasadas y los fines de semana, así que nadie puede reservar un día en que la academia está cerrada. Y a diferencia de las dos apps analizadas, que tienen partes en inglés y funciones de pago, ReservasUCT está en español, es gratuita y está hecha a la medida de la academia.

5.3 Cierre

Revisar estas aplicaciones nos dejó tranquilos en un punto: la forma en que ordenamos el proyecto, con vistas, modelo, servicios y persistencia por separado, es la misma idea que usan apps reales, solo que a menor escala. También nos dejó claro qué nos falta para acercarnos a ellas: agregar inicio de sesión y pasar del JSON local a una base de datos compartida, porque en un sistema de reservas todos tienen que ver la misma información.

Bibliografía y fuentes consultadas

Calendly. (s.f.). *Calendly: Free online appointment scheduling software*. https://calendly.com

Calendly. (s.f.). *Calendly Help Center*. https://help.calendly.com

Google. (s.f.). *Firebase Crashlytics*. https://firebase.google.com/docs/crashlytics

Kivy. (s.f.). *Kivy documentation*. https://kivy.org/doc/stable/

KivyMD. (s.f.). *KivyMD documentation*. https://kivymd.readthedocs.io

Python Software Foundation. (s.f.). *json — JSON encoder and decoder*. https://docs.python.org/3/library/json.html

Skedda. (s.f.). *Skedda: Space booking software*. https://www.skedda.com

Skedda. (s.f.). *Skedda Help Center*. https://support.skedda.com

Universidad Católica de Temuco. (2026). *Desarrollo Móvil, Clase 23: Patrón MVC y separación de responsabilidades* [Material de clase].

Bitácora de uso de IA

Usamos IA como apoyo para ordenar la estructura del informe según la pauta y para revisar la redacción. La elección de las aplicaciones, el análisis de nuestro propio código y las conclusiones los revisamos y ajustamos como equipo.